

データフロー比較図

作成日: 2026-05-25
更新日: 2026-05-31
用途: 外部ft_irc実装とのデータフロー比較
対象: IRC_torinoue, barimehdi77, itsYakub, Ala-Na, ft_IRC-InternetRelayChat-

比較サマリ

要素	IRC_torinoue	barimehdi77	itsYakub	Ala-Na	ft_IRC-InternetRelayChat-
受信バッファ位置	Connection	Server (map)	Client	Server	Client
送信バッファ	✔ Connection	✘ なし	✘ なし	✘ なし	✔ Client
CommandResult	✔	✘	✘	✘	✘
POLLOUT対応	✔	✘	✘	✘	✔
即時send()	✘	✔	✔	✔	✘
Parser分離	✔	Request	内包	Command	CommandHandler内

結論

IO堅牢性（非ブロッキング送信）の実装パターンが2系統ある。

パターン	該当	特徴
Connection + CommandResult	IRC_torinoue（設計）	層間データ型が明示、返信を一括適用
Client内バッファ + enablePollout	ft_IRC-InternetRelayChat-	実装済み、CommandHandlerから逐次バッファ積み
即時 send()	barimehdi77, itsYakub, Ala-Na	シンプルだがブロックリスク

ft_IRC-InternetRelayChat-の位置づけ:

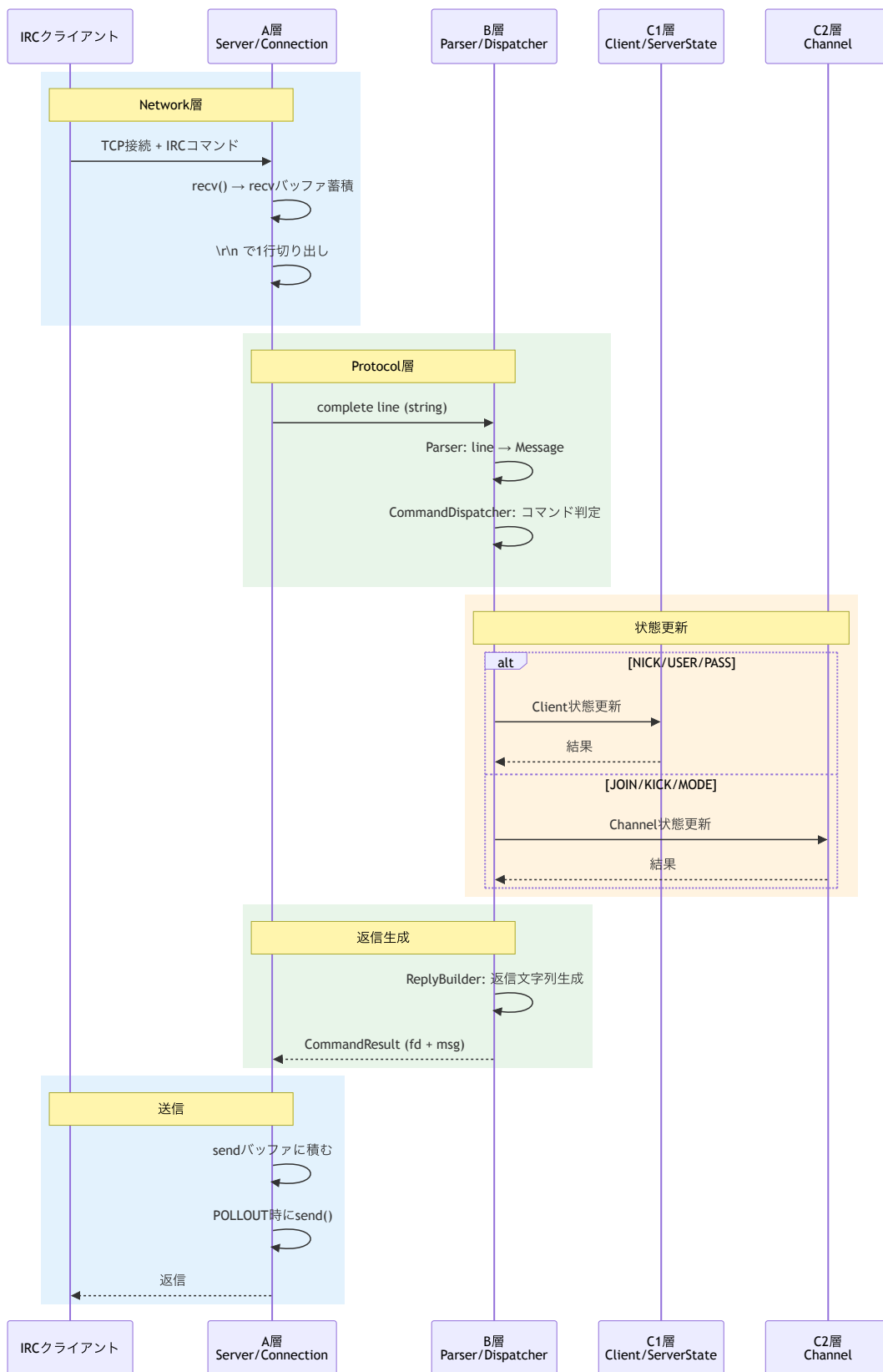
- barimehdi77 / itsYakub / Ala-Na と異なり POLLOUT + 送信バッファあり
- IRC_torinoue設計と目的は同じ（非ブロッキングIO）だが、バッファ所在と返信集約方式が異なる

色凡例（全図共通）

色	意味
● 青	Network/IO層
● 緑	Protocol/Command層
● オレンジ	Client/State層
♥ 茶	Channel層

1. IRC_torinoue（設計図）

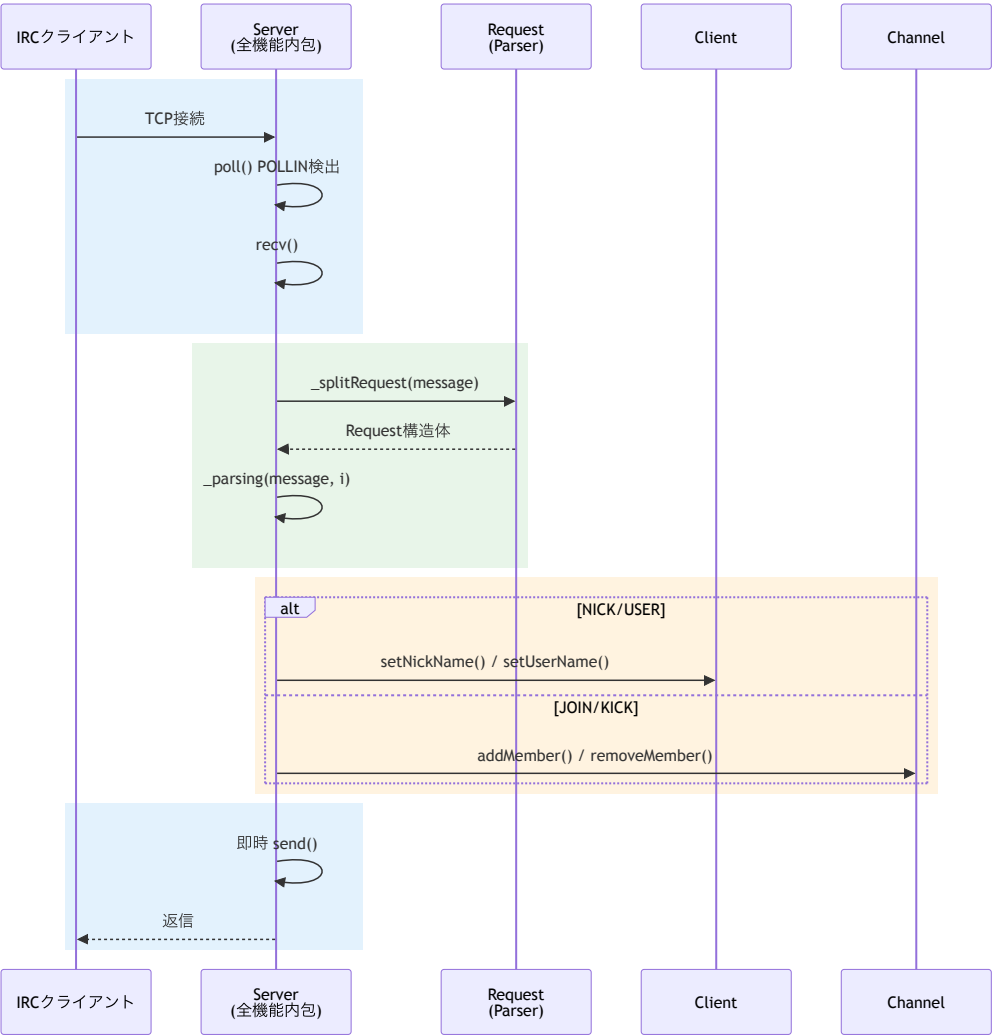
特徴: 層間で明確なデータ型を定義、CommandResultで送信を遅延

**データ型:**

- A→B: `std::string` (complete line)
- B内部: `Message` (command + params)
- B→A: `CommandResult` (replies + shouldDisconnect)

2. barimehdi77/ft_irc

特徴: Serverが全処理を内包、即時send()



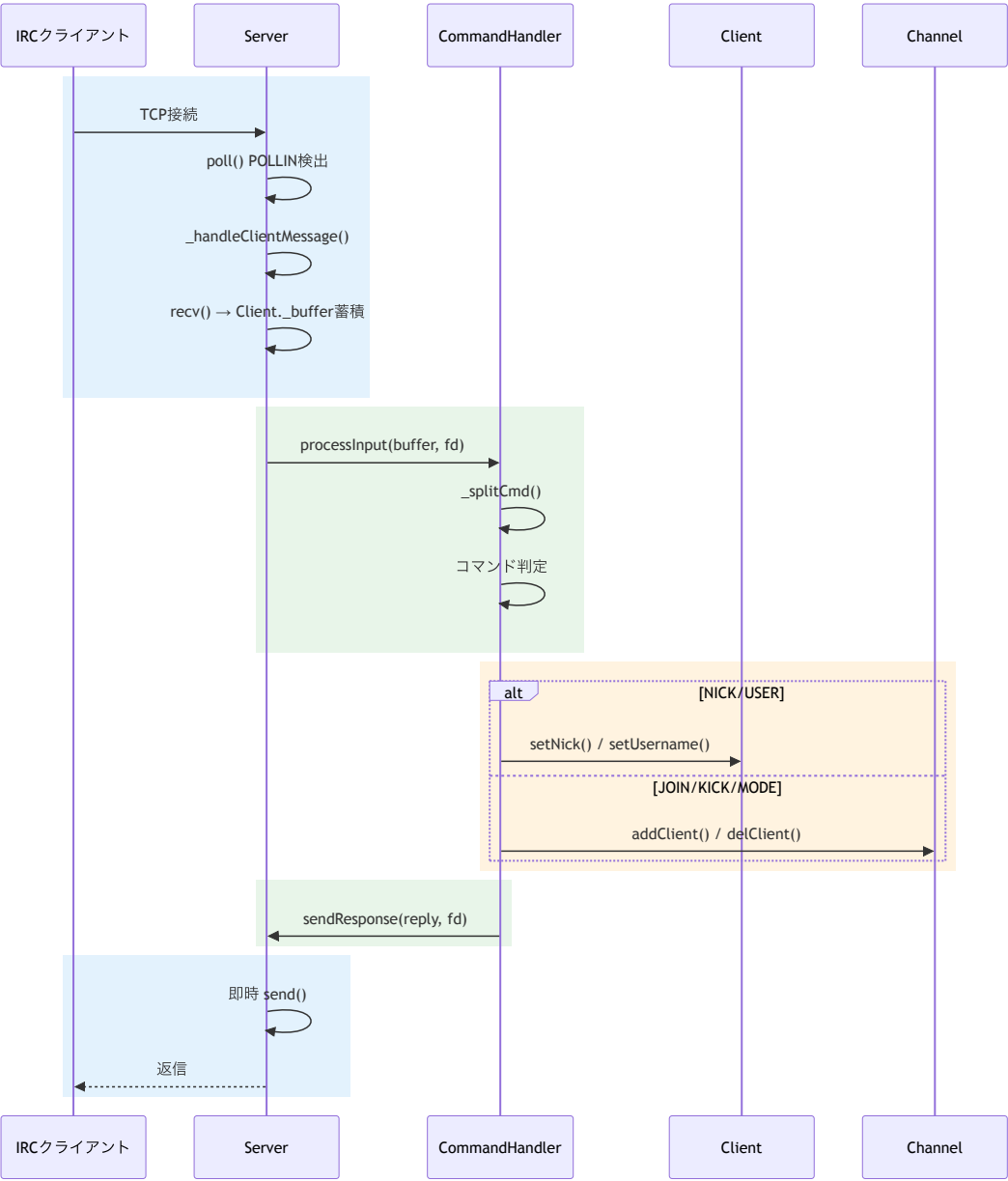
データ型:

- 内部: Request (command + args)
- 送信: 即時 send() (バッファリングなし)

問題点: send() がブロックする可能性あり

3. itsYakub/42-ft_irc

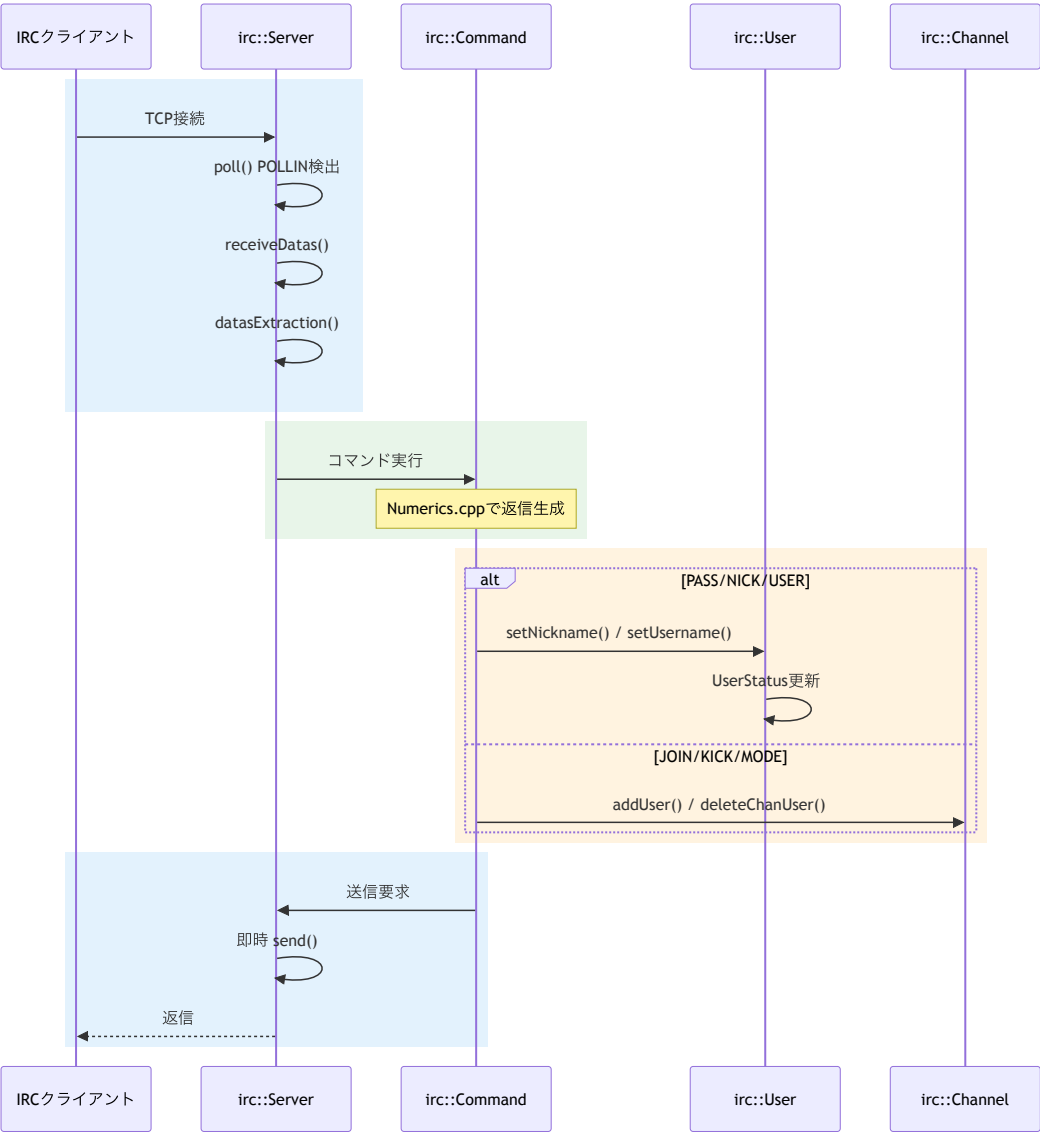
特徴: CommandHandler分離、直接send()



- データ型:
- Client内: `_buffer` (受信バッファをClient内に保持)
 - 送信: `Server::sendResponse()` 経由で即時 `send()`

4. Ala-Na/ft_irc

特徴: namespace使用、UserStatus enum、設定ファイル対応



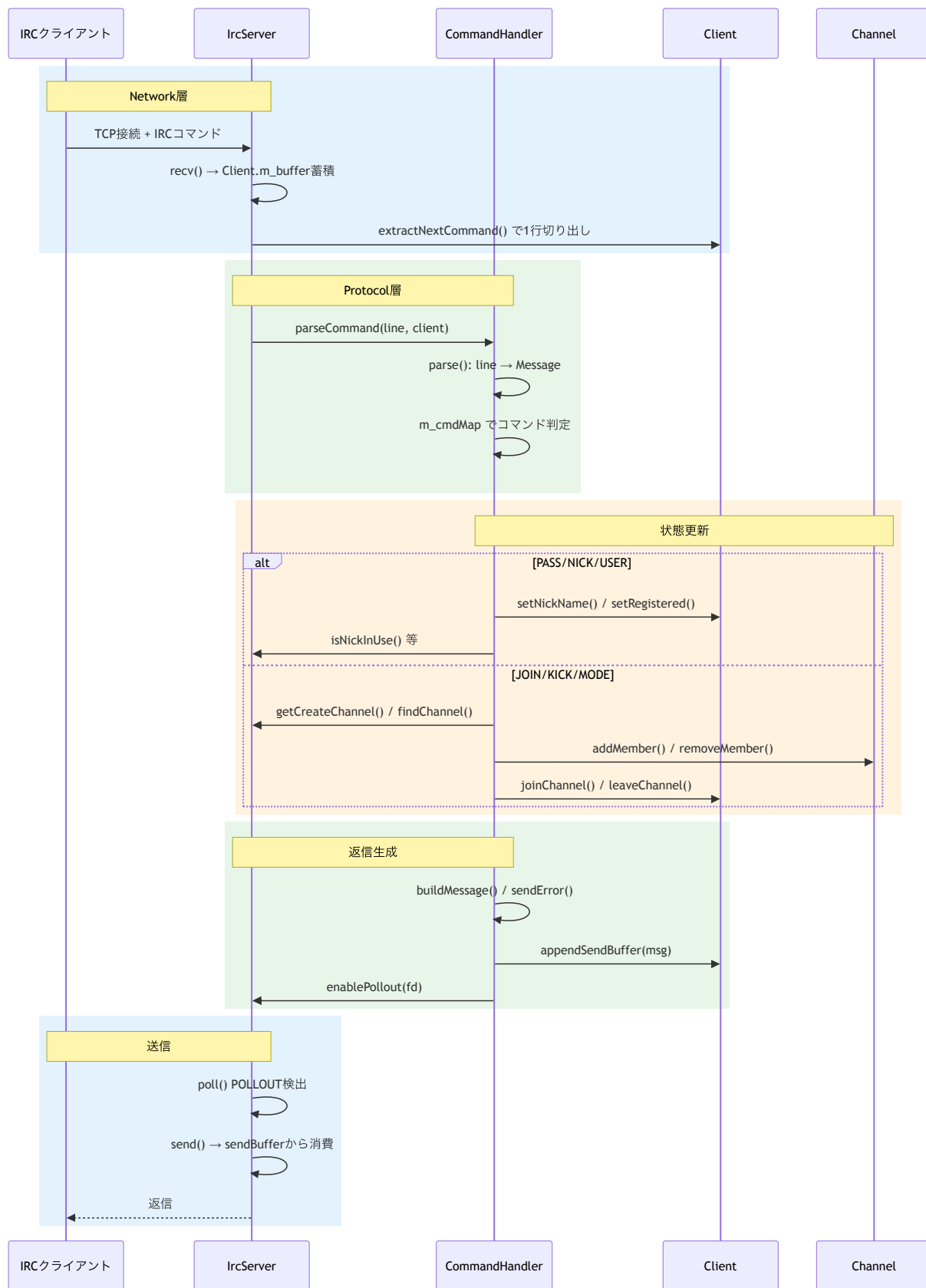
データ型:

- `UserStatus` enum: PASS → NICK → USER → REGISTERED
- 送信: 即時 `send()`

5. ft_IRC-InternetRelayChat-

特徴: Client内バッファ、CommandHandlerが解析～返信生成まで担当、送信バッファ+POLLOUT

Mandatory Part中心。Bot等の拡張は図から除外。



データ型:

- recv: `Client.m_buffer` 内で `\r\n` 切り出し
- 内部: `CommandHandler::Message` (prefix + command + params + trailing)
- 送信: `Client.m_sendBuffer` + `enablePollout()` (CommandResult相当の集約型なし)

IRC_torinoue設計との差分:

- 受信バッファ位置: `Connection` vs `Client`
- 返信の集約: `CommandResult` vs コマンド処理中に逐次 `appendSendBuffer`
- QUIT時: `removeClientFromAllChannels` をHandlerから直接呼ぶ (IRC_torinoueは `removeClient` に集約)